

# 파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

## 머릿말

Note. 참고 URL

[https://github.com/seejokim1/NA\\_with\\_python](https://github.com/seejokim1/NA_with_python)

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

# Contents

|       |                                           |   |
|-------|-------------------------------------------|---|
| 1     | 파이썬의 이해                                   | 1 |
| 2     | 비선형 방정식 구하기                               | 2 |
| 3     | 선형연립방정식 수치해법                              | 3 |
| 4     | 수치미분 (Numerical Differentiation)          | 4 |
| 4.1   | 수치미분의 개요                                  | 4 |
| 4.1.1 | 테일러 급수 (Taylor Series)                    | 4 |
| 4.1.2 | 수치미분의 개요                                  | 5 |
| 4.2   | 전진, 후진, 중심차분, Richardson 외삽법              | 6 |
| 4.2.1 | 전진차분 (Forward Difference)                 | 6 |
| 4.2.2 | 후진차분 (Backward Difference)                | 6 |
| 4.2.3 | 중심차분 (Central Difference)                 | 6 |
| 4.2.4 | Richardson 외삽법 (Richardson Extrapolation) | 7 |
| 4.2.5 | 오차 해석                                     | 7 |
| 4.2.6 | 수치미분 코드 구현                                | 8 |
| 4.3   | 연습문제                                      | 8 |

# Chapter 1

## 파이썬의 이해

## Chapter 2

### 비선형 방정식 구하기

## Chapter 3

### 선형연립방정식 수치해법

# Chapter 4

## 수치미분 (Numerical Differentiation)

### 4.1 수치미분의 개요

#### 4.1.1 테일러 급수 (Taylor Series)

##### 1. 기본 개념

미분 가능한 함수  $f(t)$ 를 다항식으로 근사하는 방법이다. 복잡한 함수를 단순한 다항식 형태로 표현하여 계산을 쉽게 한다.

---

##### 2. 중심점 $a$ 에서의 테일러 급수

$$f(t) = f(a) + f'(a)(t-a) + \frac{f''(a)}{2!}(t-a)^2 + \frac{f^{(3)}(a)}{3!}(t-a)^3 + \dots$$

---

##### 3. Maclaurin 급수 ( $a = 0$ )

$$f(t) = f(0) + f'(0)t + \frac{f''(0)}{2!}t^2 + \frac{f^{(3)}(0)}{3!}t^3 + \dots$$

---

##### 4. 예: $e^t$ 의 전개

$$e^t = 1 + t + \frac{t^2}{2!} + \frac{t^3}{3!} + \dots$$

---

##### 5. Python 예제 코드

```
import numpy as np
import matplotlib.pyplot as plt

# x 값 정의 및 원래 함수 계산
x = np.arange(0, 3.01, 0.01)
y = np.exp(x)
```

```
# 원래 함수 그래프
plt.plot(x, y, 'k', linewidth=3, label='y = exp(x)')

plt.axis([0, 3, 0, 10])
plt.grid(True)
plt.show()
```

실습예제: [https://github.com/seejokim1/NA\\_with\\_python/blob/main/chapter04/section\\_04\\_01\\_01\\_taylor\\_series\\_exp.py](https://github.com/seejokim1/NA_with_python/blob/main/chapter04/section_04_01_01_taylor_series_exp.py)

### 4.1.2 수치미분의 개요

수치미분(Numerical Differentiation)은 함수를 직접 미분하지 않고 함수값을 이용하여 미분을 근사적으로 계산하는 방법이다. 해석적 미분이 어려운 경우나, 실험 데이터로부터 미분값을 추정할 때 사용된다.

#### 미분의 정의

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (4.1.2-1)$$

수치미분에서는  $h$ 를 매우 작은 값으로 설정하여 위 식을 근사적으로 계산한다.

#### 1. 전진 차분 (Forward Difference)

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad O(h) \quad (4.1)$$

#### 2. 후진 차분 (Backward Difference)

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \quad O(h) \quad (4.2)$$

#### 3. 중심 차분 (Central Difference)

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad O(h^2) \quad (4.3)$$

중심 차분은 전진/후진 차분보다 높은 정확도를 가지므로 일반적으로 더 많이 사용된다.

#### 테일러 급수와의 관계

수치미분 공식은 테일러 급수 전개로부터 유도된다. 차분 공식의 정확도는 테일러 급수에서 남는 오차항에 의해 결정된다.

예를 들어,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \dots$$

이를 이용하면 각 차분 공식의 오차 차수를 분석할 수 있다.

#### 요약

- 수치미분은 함수값을 이용한 미분 근사 방법이다.



- 전진/후진 차분은  $O(h)$  정확도.
- 중심 차분은  $O(h^2)$  정확도로 더 정확하다.
- 모든 차분 공식은 테일러 급수 전개로부터 유도된다.

수치미분은 함수의 해석적 도함수를 구하기 어려운 경우, 함수값을 이용하여 근사적으로 미분값을 계산하는 방법이다.

대표적으로 다음과 같은 방법이 있다:

- 전진 차분 (Forward difference)
- 후진 차분 (Backward difference)
- 중심 차분 (Central difference)
- Richardson 외삽법

## 4.2 전진, 후진, 중심차분, Richardson 외삽법

### 4.2.1 전진차분 (Forward Difference)

테일러 전개로부터 유도하면:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (4.4)$$

오차는  $O(h)$ 이다.

```
def forward_difference(f, x, h):
    return (f(x + h) - f(x)) / h
```

### 4.2.2 후진차분 (Backward Difference)

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \quad (4.5)$$

오차는  $O(h)$ 이다.

```
def backward_difference(f, x, h):
    return (f(x) - f(x - h)) / h
```

### 4.2.3 중심차분 (Central Difference)

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (4.6)$$

오차는  $O(h^2)$ 이다.

```
def central_difference(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)
```

### 4.2.4 Richardson 외삽법 (Richardson Extrapolation)

Richardson 외삽법은 서로 다른 간격  $h$ 와  $h/2$ 에서 계산한 근사값을 결합하여 더 높은 정확도의 값을 얻는 방법이다. 테일러 급수 전개를 이용하여 낮은 차수의 오차항을 제거한다.

#### 1. 기본 개념

어떤 수치 근사값  $R(h)$ 가 다음과 같이 표현된다고 하자:

$$R(h) = f + c_1 h^2 + c_2 h^4 + O(h^6) \quad (4.2.4-1)$$

간격을 절반으로 줄이면,

$$R\left(\frac{h}{2}\right) = f + c_1 \left(\frac{h}{2}\right)^2 + c_2 \left(\frac{h}{2}\right)^4 + O(h^6) \quad (4.2.4-2)$$

#### 2. 오차 제거 공식

위 두 식을 조합하면  $O(h^2)$  항을 제거할 수 있다:

$$R_{\text{extrapolated}} = \frac{4R(h/2) - R(h)}{3} \quad (4.2.4-3)$$

따라서,

$$R_{\text{extrapolated}} = f + O(h^4) \quad (4.2.4-4)$$

즉, 정확도가  $O(h^2)$ 에서  $O(h^4)$ 로 향상된다.

#### 3. 미분 계산에의 적용

수치미분에 적용하면,

$$f'(x) \approx \frac{4f'(x; h/2) - f'(x; h)}{3} \quad (4.2.4-5)$$

#### 요약

- 서로 다른 간격의 근사값을 조합하여 오차항을 제거한다.
- $O(h^2)$  정확도를  $O(h^4)$ 로 향상시킬 수 있다.
- 테일러 급수 기반 오차 분석에 의해 유도된다.
- 수치미분, 적분 등 다양한 수치해석 문제에 적용된다.

### 4.2.5 오차 해석

전진/후진 차분:

$$O(h)$$

중심차분:

$$O(h^2)$$

Richardson 외삽:

$$O(h^4)$$

반올림 오차와 절단 오차의 균형이 중요하다.

### 4.2.6 수치미분 코드 구현

실습예제: [https://github.com/seejokim1/NA\\_with\\_python/blob/main/chapter04/section\\_04\\_02\\_06\\_numerical\\_derivative.py](https://github.com/seejokim1/NA_with_python/blob/main/chapter04/section_04_02_06_numerical_derivative.py)

```
import numpy as np

def f(x):
    return np.sin(x)

x = 1.0
h = 0.01

print("Forward:", forward_difference(f, x, h))
print("Backward:", backward_difference(f, x, h))
print("Central:", central_difference(f, x, h))
print("Richardson:", richardson(f, x, h))

print("Exact:", np.cos(x))
```

## 4.3 연습문제